Cardiff Racing Driverless Design Progress Report: Cone Detection, Route Planning, and Simulation

Daniel Harbourne, Mohamed Binesmael, Molly Ragonnet, Matteo de Rosa, Rachael Joyce, Deeon Roy, Ahmad Amer, Bryson Kwan, Kirill Sidorov

***Abstract*— Formula Student UK, the Europe's most established educational engineering competition, in which Cardiff Racing team have earned 1st place in 2017, have recently announced the indroduction of driverless racing discipline as part of the competition. This paper reports on the current progress of the Cardiff Racing Driverless team, and discusses the software framework, the situational awareness, and the path planning components of our system.**

I. Introduction

Recently, autonomous (driverless) vehicle control has attracted substantial attention in both the academia and the industry. Driverless racing is a recently introduced discipline within the Formula Student competition. While driverless control in urban environments in now an established research field, competitive driverless racing presents a number of unique challenges and is a relatively unexplored area. Recent advances in driverless racing include the 2017 winner, ETH Zurich's *flüela* [1], TUW-Racing Edge8 [2], and Beijing Institute of Technology vehicle [3], "Junior" in the DARPA Urban Challenge [14].

This paper reports on the current progress of the Cardiff Racing Driverless team addressing this challenge. We have adopted a modular system architecture with a simulator in the loop, whose overview is shown in Fig. 1. In this paper, we address the cone detection, path planning, and simulation components of the system.

II. Cone Detection

We have implemented and evaluated several approaches for cone detection and position estimation.

First approach is the Viola-Jones cascade detector [4] which consists of several stages of cheap classifiers. A sliding window is passed through an image, and it is classified by a stage as negative (doesn't contain the object) or positive (contains the object). If a window is classified as negative, it is rejected and the window slides to the next location. If it is classified as positive, the window is passed to the next stage. The detector output a positive detection if the window is classified positive by the final stage. Early rejection in the cascade lends to very fast operation. We have experimented with three types of features: Haar features, histograms of oriented gradients (HoG), and local binary patterns (LBP).

The second approach is the region-proposal CNN (RCNN) [5] which works by using a traditional computer vision technique called selective search to identify regions of interest (ROIs), then applying a convolutional neural network to each ROI to classify the image contained within that region. A typical image will contain about 2000 ROIs, which each need to be classified, making this process time-consuming.

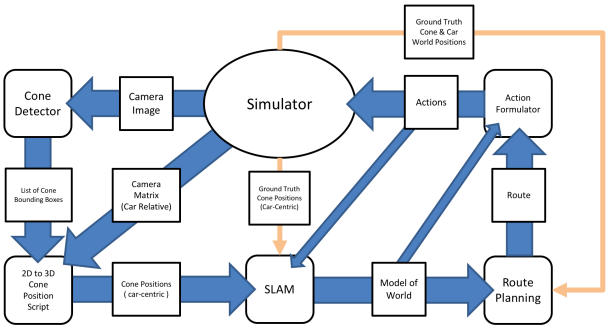


Fig. 1. System overview.

Finally, the Fast R-CNN technique [6] is similar to the R-CNN technique in that a CNN is used for classification of regions and for bounding box regression. However, unlike in R-CNNs, in Fast R-CNNs the image is passed through the CNN once, creating one large feature map. Regions of interest are then selected from this feature map and each one is passed through layers which then output bounding boxes. The main benefit is that it offers an end-to-end approach which allows for training to take place across the entire network and the forward pass time is faster.

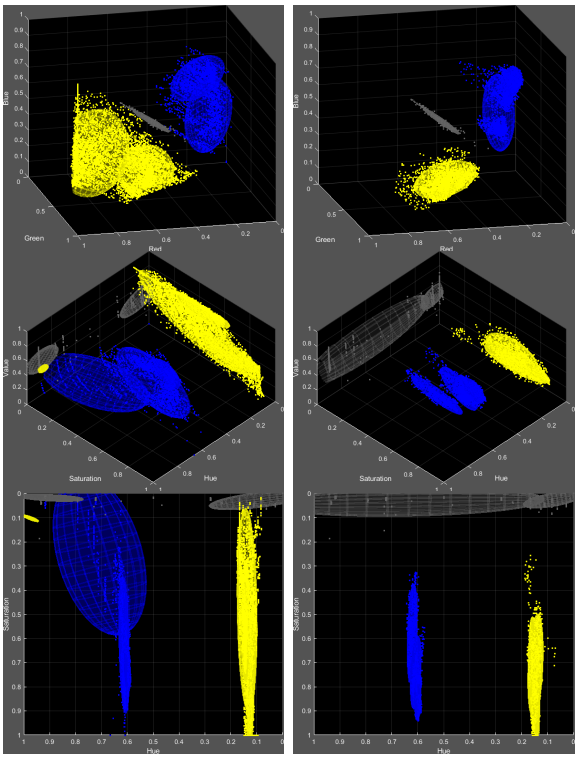
We have manually annotated 1088 images (988 in the training set, and 100 in testing), containing 3106 exemplars of cones (2395 in the training set, 711 in testing). A further 3219 negative images, including of race tracks, were used for training of the cascade detector.

Our evaluation (see Table I) shows that in this scenario the traditional computer vision techniques perform better. We hypothesise that this is due to the regular shape of the cones, which lends itself well to the feature detection techniques, as well as the relatively small number of examples.

Cone detection with cascade detector is done on grayscale values only. We further employ a statistical colour model which serves two purposes: to differentiate blue cones from yellow ones and to further filter false positive detections. For this we model the distribution of cone pixel colours as

TABLE I. Comparison of the cone detection techniques. $\mathcal{T}$ is the intersection over union acceptance threshold between the detected and the ground truth bounding boxes.

Technique	$\mathcal{T}$	Time, s/im	Precision	Recall	Avg. Prec.
Haar	0.5	0.099	0.6297	0.6825	0.4493
Haar	0.3		0.7901	0.8564	0.7287
HOG	0.5	0.165	0.5122	0.6217	0.3781
HOG	0.3		0.8625	0.7581	0.5724
LBP	0.5	0.092	0.5235	0.6259	0.3512
LBP	0.5		0.6882	0.8228	0.6393
RCNN	0.5	4.442	0.6777	0.3727	0.3052
RCNN	0.3		0.8286	0.4557	0.4413
Fast-RCNN	0.5	0.703	0.2742	0.1674	0.0507
Fast-RCNN	0.3		0.5968		0.2583



AMZ data StreetDrone data

Fig. 2. Colour model. Ellipsoids delineate one standard deviation of the Gaussians in the mixture. *Top:* RGB space; *Middle and bottom:* HSV space.

a Gaussian mixture model (GMM) which has the form [7]

$$p(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i G(\mathbf{x}, \mu_i, \mathbf{C}_i), \tag{1}$$

where n is the number of Gaussians in the mixture, μ are the centres of the Gaussians, $\mathbf{C}_i$ are the corresponding covariance matrices, and α_i are the prior probabilities. The parameters of the GMM are estimated using the well-known EM algorithm [7], [8]. GMM gives a degree of invariance to changes in colour of the cones. Working in HSV colour space gives a further degree of invariance to global illumination. The colour models may need to be adjusted for a specific camera. Figure 2 show examples of GMM colour models in RGB and HSV colour spaces.

The filtering and classification of cones by colour is performed as shown in Algorithm 1, which estimates for each candidate bounding box produced by the detector, how well it matches either of the colour models. The example results of cone detection and localisation are shown in Fig. 3.

III. Localization and Mapping

Crucial to the situational awareness of the autonomous racing car is the ability to estimate its state given the observed environment, while at the same time building up the map of the environment, the paradigm known as Simultaneous Localization and Mapping (SLAM). This is currently the weakest part of our project, with more work needed before out SLAM sub-system is robust enough for FS-AI.

Algorithm 1 Filtering false-positives with colour model

Require: Bounding box W, mixture models $(\mu^b, \mathbf{C}^b)$, $(\mu^y, \mathbf{C}^y)$ with N_b and N_y components, thresholds τ_b, τ_y, ρ_b, ρ_y.

$D^b \leftarrow 0$, $D^y \leftarrow 0$
for each pixel $(r, g, b) \in W$ **do**
  $\mathbf{x} = (h, s, v) \leftarrow$ RGB2HSV$((r, g, b))$
  $d^b_{\min} \leftarrow \inf$, $d^y_{\min} \leftarrow \inf$
  for $i \leftarrow 1$ to N_b **do**
    $d \leftarrow \sqrt{(\mathbf{x} - \mu^b_i)(\mathbf{C}_i{}^b)^{-1}(\mathbf{x} - \mu^b_i)'}$ ▷ Mahalanobis
    $d^b_{\min} \leftarrow \min(d^b_{\min}, d)$ ▷ min over components
  for $i \leftarrow 1$ to N_y **do** ▷ Same for yellow
    $d \leftarrow \sqrt{(\mathbf{x} - \mu^y_i)(\mathbf{C}_i{}^y)^{-1}(\mathbf{x} - \mu^y_i)'}$
    $d^y_{\min} \leftarrow \min(d^y_{\min}, d)$
  if $d^b_{\min} > \tau_b$ **then** $D^b \leftarrow D^b + 1$ ▷ Count outliers
  if $d^y_{\min} > \tau_y$ **then** $D^y \leftarrow D^y + 1$
  if $D^b/|W| < \rho_b$ **and** $D^y/|W| < \rho_y$ **then**
    if $D^b > D^y$ **then return** FALSE_POS_BLUE
    else return FALSE_POS_YELLOW
  else
    if $D^b > D^y$ **then return** TRUE_POS_BLUE
    else return TRUE_POS_YELLOW

We have investigated a number of SLAM techniques to build a map of the environment from cone positions: EKF-SLAM [9], ORB-SLAM [10], and LSD-SLAM [11]. Experimentally, we observed poor reliability with a monocular camera (our current sensor assumption) and a relatively featureless racing landscape.

IV. Motion Planning

Assuming the boundaries of the track have been estimated by the SLAM sub-system, we proceed to discuss the problem of choosing the optimal path for the vehicle to follow. For a recent review of motion planning techniques for autonomous vehicles the reader is referred to [12].

One popular approach to the path planning problem is the discretisation of the state space such that standard search algorithms like A* and D* can be used to find the shortest path from the current to the goal state. In the scenario of autonomous driving the problem with these algorithms is that they do not consider the non-holonomic nature of the vehicle. We, therefore, employ an adaptation of A*, termed Hybrid A* [15], which can deal with the continuous nature of the search space. Hybrid A* was used effectively for getting a car to navigate and perform complex manœuvres in urban environments, such as reversing into parking spots [13], and for successfully driving Stanfords vehicle "Junior" in the DARPA Urban Challenge [14]. Figure Fig. 4 shows the comparison between A*, D*, and Hybrid A* search strategies.

Algorithm outline: Hybrid A* requires spatially discretising the world (track) into (x, y) cells forming a grid (represented as a bitmap, with 0 representing traversable space and 1s the obstacles, *e.g.* track boundaries). This bitmap is expected to be produced by rasterising the track boundaries estimated by the SLAM sub-system. The discretised map which is then used to close off search space when a cell is expanded by a node, just like standard A*, but it is possible for multiple nodes lie within an expanded

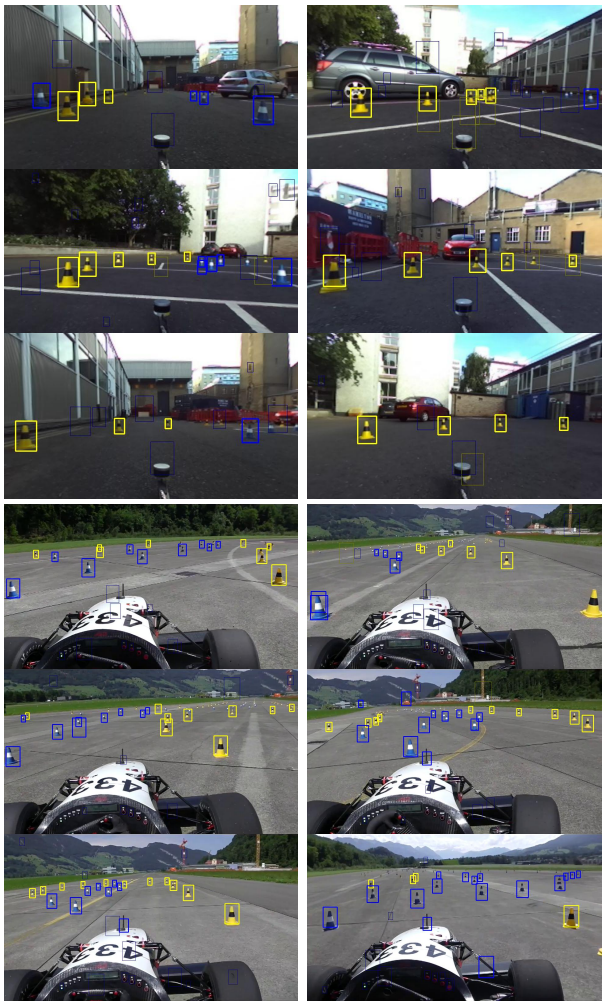


Fig. 3. Examples of cone detection. Thick likes indicate firm detections, thin lines indicate false detections discarded by the colour model. *Top:* StreetDrone data; *bottom:* AMZ data.

cell. An open list O is used to store nodes that have been expanded and might need further exploration. Once a node is conclusively processed it is placed in the closed list C. Nodes are modeled with coordinates and heading (x, y, θ). The starting position and orientation of the car correspond to the root node. Given a node, a set of new nodes can be calculated from a pre-defined traveling distance and a set of steering angles, also known as motion primitives. The number of new nodes to be explored is a tuneable parameter. The new nodes keep a pointer to their parent node p and are then added to an open list. The cost $f(n) = g(n) + h(n)$ is computed for each node, where $g(n)$ is the cost required to reach the node, in our case the distance travelled, and $h(n)$ is the heuristic which is our estimate of the minimum cost from the goal. The node with the lowest $f(n)$ is chosen from the open list and the process from calculating motion primitives is repeated until the shortest path to the goal is found.

We have experimented with Euclidean and Dubins path length heuristics for $h(n)$. Dubins path is the shortest curve that connects two points in $\mathbb{R}^2$, with the constraints on the curvature of the path, the tangents of the initial and final points of the path given that the vehicle that is travelling along the path cannot reverse. (If the vehicle is able to reverse, then the Reeds-Shepp model can be used.) Dubins path does not does not take obstacles or barriers into consideration. Euclidian distance is faster to calculate, but using it affects the quality of the produced path (see Fig. 6).

As with ordinary A*, the open list keeps track of all the nodes that have been expanded, the node with the lowest $f(n)$ is removed from the list and used to further calculate the motion primitives. For performance reasons, we use the binary min-heap data structure with $O(1)$ time complexity for finding the minimum $f(n)$ node, $O(\log n)$ for deleting and inserting. There are occasions where nodes have the same $f(n)$ we want to perform a tie-break, this is where we choose the node with the highest $g(n)$ from the two this is so that it encourages exploration that is closer towards reaching the goal.

Motion Primitives: The path is composed of arcs (motion primitives) subject to the following constraints:

- The distance travelled d has to be enough to exit the cell. For a cell of size s, it must be the case that $d \geq \sqrt{2} \cdot s$.
- The curvature radius r is constrained by the maximum vehicle steering angle α.

For a vehicle of length l traveling along a motion primitive of length d with a steering angle α, the turning angle $\beta = d/(l \tan \alpha)$ and the curvature radius of the turn is $r = d/\beta$. The update to the vehicle state (x, y, θ) is calculated as follows:

$$\Delta x = x - r \sin \theta, \qquad x_{\text{new}} = \Delta x + r \sin(\theta + \beta), \quad (2)$$

$$\Delta y = y - r \cos \theta, \qquad y_{\text{new}} = \Delta y + r \cos(\theta + \beta), \quad (3)$$

$$\theta_{\text{new}} = \theta + \beta \pmod{2\pi}, \quad (4)$$

The end point of the motion primitive is then used to create a new node $N = (x_{\text{new}}, y_{\text{new}}, \theta_{\text{new}})$. Each motion primitive created is checked for collisions, which involves checking if any points lie within a specified radius from the front left, front right, and center point of the vehicle.

We illustrate the operation of the Hybrid A* path planner in Figs. 5 and 6.

Post-optimisation: Due to the discretization of the steering angle range, it is unlikely for Hybrid A* to find the global optimum, but the solution is likely to be within the neighbourhood of it. This typically also leads to producing a path that is sub-optimal and not smooth, where unnecessary turns are made along the path. To improve the produced path, this project will aim to post-process the produced path by performing some local optimization and smoothing.

The path produced by the above Hybrid A*, while close to the optimum, might be sub-optimal due to discretisation. We employ a further post-optimisation step, as proposed by Dolgov *et. al* [15], to improve the length and smoothness of the path. Let the path be defined by a sequence of vertices $\mathbf{x}_i = (x_i, y_i)$, and let $\Delta \mathbf{x}_i = \mathbf{x}_i - \mathbf{x}_{i-1}$. Denote by $\mathbf{o}_i$ the location of the obstacle nearest to $\mathbf{x}_i$ and let $\Delta \phi_i$ be the change in tangential angle at $\mathbf{x}_i$. The objective function to

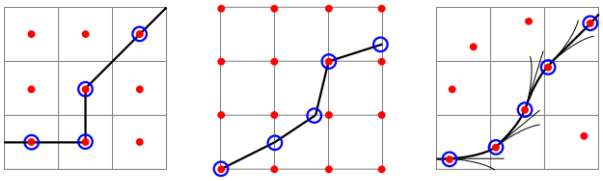


Fig. 4. Comparison of search algorithms. Nodes are represented by the red dots. *Left:* A*, *centre:* D*, *right:* hybrid-state A*. (From Dolgov *et. al* [15])

be minimised is [15]:

$$\min \leftarrow w_\rho \sum_{i=1}^{N} \rho_d(x_i, y_i) + w_o \sum_{i=1}^{N} (|\mathbf{x}_i - \mathbf{o}_i| - d_{\max})^2 + w_\kappa \sum_{i=1}^{N-1} \left(\frac{\Delta\phi_i}{|\Delta\mathbf{x}_i|} - \kappa_{\max} \right)^2 + w_s \sum_{i=1}^{N-1} (\Delta\mathbf{x}_{i+1} - \Delta\mathbf{x}_i)^2, \tag{5}$$

where ρ_d is the field of distances to the nearest obstacle, $\kappa_{\max}$ is the maximum driveable curvature of the path, and w_ρ, w_o, w_κ, w_s are tuneable weights. A slightly more sophisticated approach, based on optimisation (in terms of safety, comfort, efficiency, energy *etc.*) is adopted in [16].

V. Simulation

We have considered several simulation solutions with which to test and tune our algorithms.

TORCS is used traditionally in racing simulation but difficult to customize and interface with programmatically. Udacity (based on Unity engine) offers sophisticated graphics but only very basic mechanical modeling of the vehicle, and no direct simulation of advanced sensors (e.g. LIDAR). Finally, IPG CarMaker offers extensive realism in physical simulation, sensor modeling, and affords integration with MATLAB/Simulink. We have settled on a combination of Udacity with which to test computer vision parts of the system, and IPG CarMaker for overall system simulation.

We have modeled the platform vehicle from FSUK 2018 competition in IPG CarMaker (see Figs. 7 and 8), as well as mocked up a number of test scenarios, *e.g.* the Silverstone circuit (see Fig. 9). Some of the vehicle parameters we are using are summarised in Table II. We are employing IPG CarMaker's integration with MATLAB/Simulink to run the tests programmatically.

Work is currently underway to improve the realism of simulation, both graphically and mechanically, to bring it closer to the conditions of the FSUK competition. We intend to make our models and APIs publicly available, so that all teams can take advantage of a common, convenient, and powerful testbed.

VI. Future Work

Currently, the weakest part of the project is the SLAM sub-system, which we intend to finish in time for the 2019 competition.

A number of improvements will concern trajectory planning. At present, the trajectories are planned with no regard for velocity profiles. For optimal racing performance, it is necessary that the trajectory and the velocity be optimised

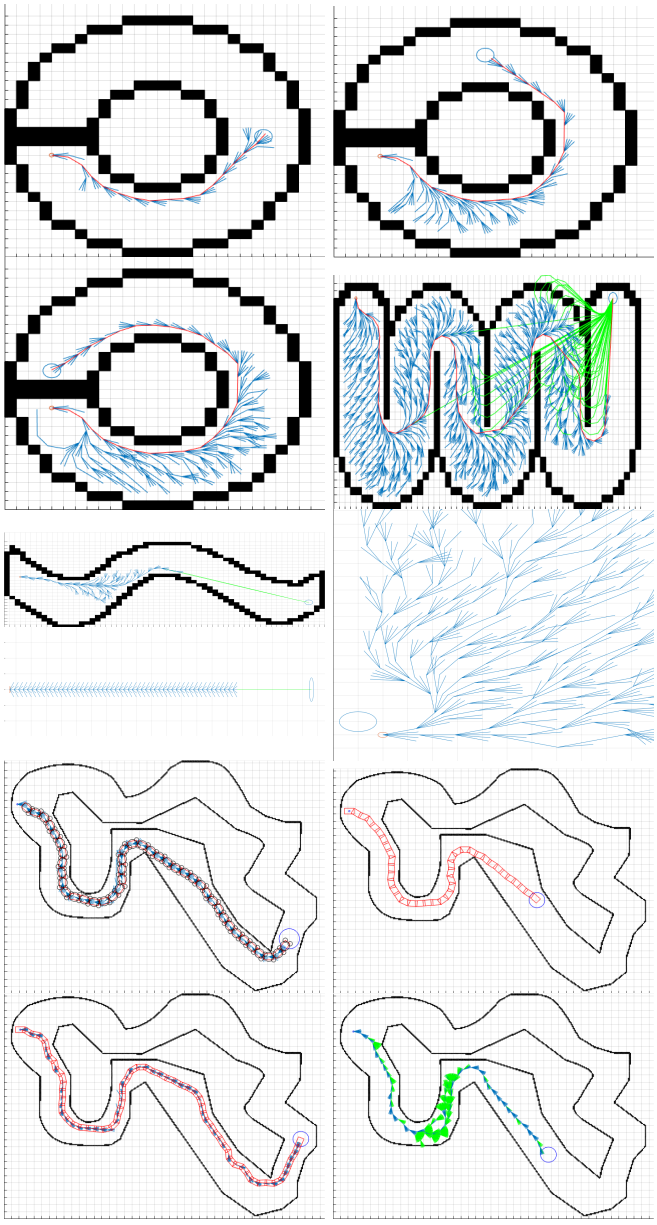


Fig. 5. Illustration of motion planning with Hybrid A* and driveable motion primitives.

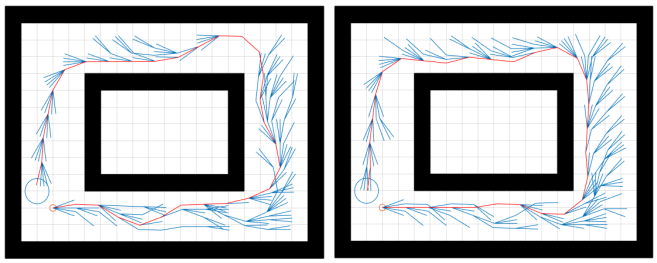


Fig. 6. The effect of introducing additional penalty for large steering angles to the Euclidean heuristic.

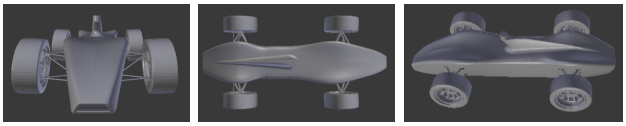


Fig. 7. CAD models used in the simulation.

TABLE II. Vehicle parameters in IPG CarMaker. In bold: measured from the platform vehicle, other parameters are estimated.

Parameter	Value	Parameter	Value	Parameter	Value
Length	**1.170 m**	Height	**0.229 m**	Body mass	**191.84 kg**
Overall mass	**308.90 kg**	Rotation (torsion)	5000 Nm/deg	Rotation (bending)	15000 Nm/deg
Front wheel carrier mass	1.67 kg	Back wheel carrier mass	1.44 kg	Wheel mass	8.71 kg
Engine mount stiffness	50000 N/m	Engine mount damping	500 Ns/m	Suspension spring stiffness	35000 N/m
Front damping (push)	2500 Ns/m	Front damping (pull)	5000 Ns/m	Buffer stiffness (push)	5000 N/m
Buffer stiffness (pull)	5000 N/m	Front stabilizer stiffness	287 N/m	Rear stabilizer stiffness	191.9 N/m
Steering pinion angle	100 rad/m	Motor power	17 kW	Motor torque	55 Nm
Aerodynamic reference area	2 m^2	Aerodynamic reference length	1.3 m		

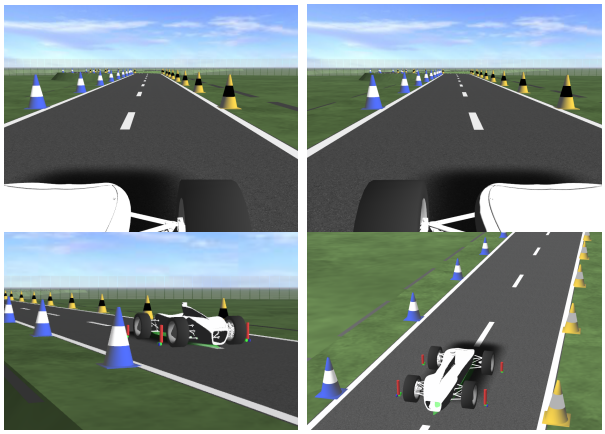


Fig. 8. Camera views from the IPG CarMaker simulator.

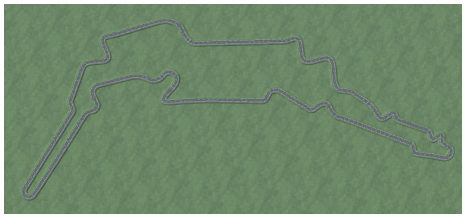


Fig. 9. Silverstone Formula Student 2015 track modeled in IPG.

jointly. This is a challenging problem, and the approach we are currently implementing involves iterative optimisation of both, and is due to [16].

As far as the sensors are concerned, we have so far concentrated only on the hardest scenario, whereby the vehicle is equipped only with a monocular camera. Future work will include techniques for cone detection from LIDAR data, a stereopair of cameras, and fusion of these.

Further, given the stellar success of deep reinforcement learning, in some challenging tasks achieving better than human performance [17], we intend to investigate (in a simulated environment) whether reinforcement learning can be applied to driverless racing control, and compare it with traditional control techniques. We further intend to compare the performance of the existing control sub-systems (including the one based on reinforcement learning) with that of a human driver, and explore whether any heuristics, derived from observing human control, can be employed to improve automatic control.

A path following controller (*e.g.* [3]) remains to be implemented. Optimal tuning of path following controller will involve accurate modeling of drive-by-wire controls, such as throttle, steering, and braking-by-wire.

Finally, integration of all components into a complete system presents a non-trivial task. We are currently transitioning to using the Robot Operating System (ROS) as a platform for system integration, modularisation, and communication.

ACKNOWLEDGEMENTS

The authors would like to thank MathWorks for providing competition MATLAB licenses and IPG Automotive for providing the IPG CarMaker simulator.

REFERENCES

[1] M. de la Iglesia Valls, H. F. C. Hendrikx *et al.*, "Design of an autonomous racecar: Perception, state estimation and system integration," *CoRR*, vol. abs/1804.03252, 2018.

[2] M. Zeilinger, R. Hauk *et al.*, "Design of an Autonomous Race Car for the Formula Student Driverless (FSD)," in *Proc. OAGM and ARW-17*, Vienna, Austria, May 2017, pp. 57–62.

[3] J. Ni and J. Hu, "Path following control for autonomous formula racecar: Autonomous formula student competition," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, June 2017, pp. 1835–1840.

[4] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. CVPR 2001*, vol. 1, Apr. 2001, pp. I–511–I–518.

[5] R. B. Girshick, J. Donahue *et al.*, "Rich feature hierarchies for accurate object detection and semantic segmentation," *CoRR*, vol. abs/1311.2524, 2013.

[6] R. B. Girshick, "Fast R-CNN," *CoRR*, vol. abs/1504.08083, 2015.

[7] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. Wiley-Interscience, 2000.

[8] W. H. Press, S. A. Teukolsky *et al.*, *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. New York, NY, USA: Cambridge University Press, 2007.

[9] M. Macsek, "Mobile robotics: EKF-SLAM using visual markers for vehicle pose estimation," Master's thesis, Vienna University of Technology, 2016.

[10] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: a versatile and accurate monocular SLAM system," *CoRR*, vol. abs/1502.00956, 2015. [Online]. Available: http://arxiv.org/abs/1502.00956

[11] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *European Conference on Computer Vision (ECCV)*, September 2014.

[12] B. Paden, M. Cáp *et al.*, "A survey of motion planning and control techniques for self-driving urban vehicles," *CoRR*, vol. abs/1604.07446, 2016.

[13] J. Petereit, T. Emter *et al.*, "Application of hybrid A* to an autonomous mobile robot for path planning in unstructured outdoor environments," in *ROBOTIK*, 2012.

[14] M. Montemerlo, J. Becker *et al.*, "Junior: The Stanford entry in the urban challenge," *Journal of Field Robotics*, 2008.

[15] D. Dolgov, S. Thrun *et al.*, "Practical search techniques in path planning for autonomous driving," in *Proc. STAIR-08*, June 2008.

[16] W. Xu, J. Wei *et al.*, "A real-time motion planner with trajectory optimization for autonomous vehicles," in *2012 IEEE International Conference on Robotics and Automation*, May 2012, pp. 2061–2067.

[17] V. Mnih, K. Kavukcuoglu *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.